PromptForge

提示词锻造台



让 AI 开发从「反复试错」变成「一次选对」

提示词编译器 · 多智能体评审会 · 机机联动闭环

Coding 赛道 | deforoout 队 | 首届 openJiuwen 智能体创新大赛

PromptForge： 把一句话需求编译成可验证的方案

不是改提示词，而是做一次有证据的技术决策。

创意描述 （<500 字）

PromptForge 是 AI 开发者的**方案编译器 + 多智能体评审会**。

输入一句话，先编译成含 **6 条可执行验收标准** 的 SRD；再生成 3~5 个“假设不同”的候选提示词，调用多模型同题执行；测试、静态分析、性能等证据回读后，由异构评委盲评， TOPSIS/Pareto 选出最优，并给出成本账单与可回放交付包。

它解决三个问题：需求说不清、方案选不对、结果没验证。与单模型 self-refine 不同，它用执行证据替代主观自评；与提示词工具不同，它从“改句子”升级为“做决策”。

机机联动：执行机回传 Evidence，评审机给 Verdict，规划机决定下一轮——不是让模型聊天。

已经真实跑过： A raw 3/6 vs B compiled 6/6

同一 deepseek-flash； ¥0.0096 vs ¥0.0055； 证据在 runs/demo-004 。

和现有方案的区别

- **提示词工具**：只改文本，不知道代码能不能跑、测试过不过
- **单模型 self-refine**：同一条思路自我修改，没有外部证据与参照系
- **多智能体辩论**：开放式聊天，成本随轮数放大，结论难审计

应用场景 （<500 字）

面向谁

独立开发者、高校学生、 2~20 人研发团队；接到一句话需求、要给现有项目加功能 / 修 Bug/ 做方案选型的人。

典型场景

从一句话想法做可运行原型；给现有 FastAPI/ 脚本项目加筛选、分页、鉴权等能力；团队评审会用“测试通过率 + 成本 + 置信度”替代“我觉得”。

价值

对个人：降低与 AI 协作的提示词门槛；对团队：评审可比较、可审计；对流程：让“选方案”第一次可复现。

所属主题 **Coding 赛道**

五个创新点

01 证据驱动的多方案决策
02 异构评委去偏 + 统计置信
03 Token 预算感知的自适应编排
04 Orchestrator + Skill 双形态
05 机机联动协议 IAOV

方案构想：一条可审计的七阶段流水线

从关键词到交付包，每一步都留下可复现的状态。
核心不是“让模型多写几版”，而是让候选方案在同一套验收与预算下竞演。

机机联动：执行结果回读，评审结论再驱动下一轮

01	02	03	04	05	06	07
需求编译	**候选组合**	**多模型执行**	**证据回读**	**异构评审**	**决策融合**	**交付回放**
关键词 → SRD	假设不同	并行生成	测试 / lint / 性能	盲评 / 换位 / 去偏	TOPSIS / Pareto	证据包 / DAG

SRD
我们到底要什么
目标 · 约束 · 验收标准 · 输出契约

生产者：需求编译 Agent

Evidence Packet
机器实际做成了什么
测试 · 编译 · 性能 · 截图 · 日志

生产者：执行机 / 沙箱

Prompt DAG
方案怎么演化来的
父代 · 变异 · 融合 · 成本

生产者：编排器

核心机制
- 证据驱动：每个候选真实执行，评委只在该证据上打分
- 异构评审：跨厂商评委 + 换序 + 一致性检验
- Token 预算：计量、剪枝、早停，预算耗尽前给当前最优
- 双形态： Orchestrator 跨模型决策 + Skill 单会话自迭代

技术依赖
- openJiuwen WorkSwarm / AgentOS： 任务规划、 Skills 、Channel
- Agent Framework + Agent System Service： 多 Agent 协同、沙箱
- DeepSeek / Qwen / Pangu： 模型网关与异构评委
- Python + pytest + Git + 华为云 AI 开发者空间

方案优势
- 从“文本优化”升级为“方案决策”，推荐能指到具体证据
- 多候选是假设不同，不是同义改写；差异有信息量
- 评审去偏 + 置信区间，不把“一个 LLM 打分”当真相
- 成本可见、可算、可停止；交付包可复现、可审计

决策规则： 硬约束过滤 → Pareto 前沿 → TOPSIS + 置信区间 → 分差在 CI 内不强行排序；输出推荐 + 次优 + 证据链 + 成本账单。

交付物： SRD / 候选提示词 / 证据包 / 评审报告 / 决策卡 / 成本账单；全部可按哈希回放。

DEMO：同一任务，两种提示策略，真实测试选优

不是演示“模型会写代码”，而是演示“选择有证据”。
任务：给 FastAPI 待办 API 增加标签筛选 + 分页；6 条验收测试 A1–A6。

候选 A raw-keyword

只给一句话需求 + baseline，不给验收测试

✗ A1 失败	✓ A2 通过	✗ A3 失败
✗ A4 失败	✓ A5 通过	✓ A6 通过

3/6

费用 ¥0.0096 | 模型 12.4s

证据：evidence/A.json

候选 B compiled-srd

SRD + 6 条验收 + 边界清单

✓ A1 通过	✓ A2 通过	✓ A3 通过
✓ A4 通过	✓ A5 通过	✓ A6 通过

6/6

费用 ¥0.0055 | 模型 5.7s

证据：evidence/B.json

关键用户路径

① **输入** 一句话需求 / 关键词 / 现有仓库

② **回显** SRD + 6 条可执行验收，用户确认

③ **生成** 3~5 个“假设不同”的候选提示词

④ **执行** 多模型并行生成 + 沙箱真跑 pytest

⑤ **评审** 证据回读 + 异构评委 + 对比看板

⑥ **决策** 推荐 + 次优 + 成本账单 + 交付包

根因对照： A 把 limit 默认写成 10、响应模型缺 limit/offset；B 用 Query(default=20, ge=1, le=100) 并返回完整字段。

决策：推荐 B （6/6，硬约束满足） A 3/6 （A1/A3/A4 失败）不满足硬约束；同等任务下 B 还更快、更便宜。

依据：pytest 原始输出 · evidence.json · token 费用 | 数据来自真实运行，不手填。

本次运行 runs/demo-004 | 2026-10-27 | deepseek-flash | 空闲时段价

交付物 CLI 一条命令 · candidates/A、B/main.py · evidence/{baseline,A,B}.json · decision.md · 60 秒录屏

代码仓库 github.com/dissill/LLM | 数据规则：图中红绿 / 分数 / 费用全部来自真实日志；不伪造、不手填。

口径： 任务一为单任务 / 单模型 A/B；第二个任务已复现（A 4/6、B 6/6，2026-10-31）；不宣称统计显著；异构评委与完整机机联动为设计目标。

机制展开：为什么这不是“让模型打个分”

补上软件工程的三件基础设施：可判定的需求、可验证的证据、可回放的版本。
每个候选必须真实执行；评委只能基于同一份证据打分；推荐必须能指到测试输出与费用。

01 需求编译 SRD

把“一句话”变成可判定的任务

- 关键词 / 口语 → 目标、约束、验收标准、输出契约
- 3~5 个澄清问题，用户确认或修改后才生成候选
- 验收标准必须可执行：测试 / 命令 / schema，不接受“看起来对”

实测 / 例子 例：给待办 API 加标签筛选 + 分页 → 6 条验收 A1–A6

02 候选组合 Portfolio

假设不同，不是措辞不同

- 3~5 个候选分别对应不同架构假设、验证策略、边界处理
- 相似度去重 + 覆盖度检查，避免“同一句话换三种说法”
- 每个候选声明执行契约：输出格式、验收命令、禁止项

实测 / 例子 例：A 只给一句话；B 给 SRD + 验收 + 边界；差异是假设

03 证据包 Evidence Packet

机器实际做成了什么

- 代码类强证据：pytest、编译、静态分析、性能、依赖扫描
- UI/ 图表 / 视频可选多模态：关键帧 + OCR/ASR + 元数据
- 每条结论带 evidence_refs 与哈希，支持离线回放与审计

实测 / 例子 例：A 3/6，失败 A1/A3/A4；每条失败都指向 pytest 用例

04 评审与决策

从评分到可解释的选择

- 异构评委：不同厂商 / 规模，匿名盲评 + 换序 + 偏差校正
- 一致性检验 Krippendorff's α；低一致性加评委或转人工
- 硬约束过滤 → Pareto → TOPSIS + 置信区间；CI 内不强行排序

实测 / 例子 例：硬约束先过滤 A；B 在质量 / 成本 / 时延上更优

双形态与单一事实源： Orchestrator 负责跨模型、多候选、可审计决策；Skill 负责单会话内低延迟自迭代；两者共享 SRD / Evidence / DAG。

预算： Token Ledger 逐次计量，候选剪枝 + 早停；预算耗尽前输出当前最优，而不是“再跑一轮看看”。

机机联动：让执行结果成为下一次决策的证据

不是让多个模型聊天，而是让规划机、执行机、评审机通过 IAOV 协议闭环。
Coding 的真相在测试、编译器和运行环境里；机机联动把“机器实际做了什么”变成可回读的结构化观察。

规划机 Planner
SRD → Intent

- 拆解任务、选择候选
- 分配预算、权限与轮数
- 决定修复 / 融合 / 停止

Intent / Action →

执行机 Executor
Action → Observation

- 编码 Agent、沙箱、CI
- 浏览器 / 设备适配器
- 回传 Evidence Packet

Observation →

评审机 Judge
Evidence → Verdict

- 规则引擎 + 异构评委
- 盲评 / 换序 / 一致性
- 输出置信度与下一步

IAOV 联动总线 Intent · Action · Observation · Verdict

每条消息带 contract_hash · trace_id · permission_scope · budget · endpoint_id；禁止只传自然语言聊天记录。

← Verdict 回到规划机 / 执行机

为什么难

- **异构接口**：每个执行体能力 / 参数不同 → Capability Manifest + Action schema
- **权限与安全**：越权 / 危险动作 → 最小权限 + 沙箱 + 审批门
- **状态与成本**：上下文漂移 / 来回成本 → 单一事实源 + Token Ledger + 早停

已经跑通的第一段

v0.1 实测：生成 → 沙箱 → pytest → Evidence Packet → 决策卡；A 3/6、B 6/6，费用 / 耗时均来自真实日志。

协议可复用：同一套 IAOV 与状态对象可接到 CI / 浏览器 / 设备执行体；不另起一套系统。

边界：不做无协议、无预算、无权限的开放式多机聊天；高风险动作必须人工确认。

联动的价值不是多几个 Agent，而是每一次执行都可回读、可审计、可复用。